SYSTEM CALL TRACER

A MINI-PROJECT REPORT

Submitted by:

SRIRAM U 231901053

VAISHAAL RAJHA T C G 231901502

In partial fulfillment of the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING
(CYBER SECURITY)

RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

An Autonomous Institute

CHENNAI

MAY 2025

BONAFIDE CERTIFICATE

Certified that this project " SYSTEM CALL TRACER" is the Bonafide work of "SRIRAM U, VAISHAAL RAJHA T C G" who carried out the project work under my supervision.

This mini project report is submitted for the viva voce examination to be held on _____

SIGNATURE

MRS V JANANEE
ASSISTANT PROFESSOR
Dept. of Computer Science and Engineering,
Rajalakshmi Engineering College
Chennai

INTERNAL EXAMINER EXTERNALEXAMINER

ACKNOWLEDGEMENT

We express our sincere thanks to our Head of the Department **Mr. Benedict J N**, for encouragement and being ever supporting force during our project work.

We also extend our sincere and hearty thanks to our internal guide **Ms. JANANEE V**, for her valuable guidance and motivation during the completion of this project.

Our sincere thanks to our family members, friends and other staff members of computer science engineering.

SRIRAM U 231901053
VAISHAAL RAJHA T C G 231901502

TABLE OF CONTENTS

CHAPTER 1
ABSTRACT

System calls serve as the critical interface between user-space applications and the operating system kernel. Monitoring and analyzing system calls provides deep insights into the behavior, performance, and security of software systems. This project introduces a System Call Tracer, a lightweight and efficient tool designed to capture and log system calls made by user-space applications in real time. The tracer aids developers, security analysts, and system administrators in debugging, performance tuning, and detecting malicious activities. By leveraging low-level hooks and kernel-level tracing mechanisms (such as ptrace or eBPF), the tool captures detailed information, including call names, arguments, return values, and timestamps. The output is presented in a human-readable format with optional filtering capabilities. This tracer supports multiple platforms and is designed with minimal overhead to ensure real-time responsiveness and low system impact. Overall, the System Call Tracer is a valuable utility for enhancing transparancy and control in morden computing environments.

CHAPTER 2
INTRODUCTION

Debugging complex software issues often requires peering beneath the surface of high-level code. System calls represent the low-level requests made by applications to the operating system, and examining them can be instrumental in diagnosing unexpected behavior, identifying resource conflicts, or pinpointing the root cause of errors. Existing debugging tools may not always provide this level of granular detail. To address this, we present SystemCallTracer, a lightweight utility aimed at providing developers and system administrators with the ability to trace and analyze system calls in real-time or for post-mortem analysis, thereby facilitating more efficient troubleshooting and problem resolution.

2.1. Key Features

- System Call Interception: The fundamental ability to intercept and record system calls made by a target process. This is the core of the tracer.
- Real-time Display: Showing the system calls as they happen, providing an immediate view of the program's interaction with the kernel.
- Timestamping: Recording the time at which each system call occurs, crucial for performance analysis and understanding the sequence of events.
- Process Identification: Clearly indicating which process (by PID) is making each system call, especially useful when tracing multiple processes or their children.
- System Call Name Resolution: Translating the raw system call numbers into human-readable names (e.g., open, read, write). This often requires mapping system call numbers to their names, which can be OS-specific.
- Argument Display: Showing the arguments passed to each system call. This provides context and details about what the system call is doing (e.g., the filename for open, the file descriptor and buffer for read).

- Return Value Display: Showing the value returned by each system call, which often indicates success or failure and provides additional information (e.g., the file descriptor returned by open, the number of bytes read by read).

CHAPTER 3
SCOPE OF PROJECT

The scope of the SystemCallTracer project encompasses the design, implementation, and basic evaluation of a command-line tool capable of intercepting, recording, and displaying system calls made by a specified user-space process on a [**Operating System**]. The tool will provide real-time tracing of system call names, arguments, and return values, along with timestamps and process identification.

This project aims to develop a more feature-rich SystemCallTracer for [**OS**]. In addition to the basic functionality of intercepting and displaying system calls with their names, PIDs, and arguments in real-time, this tool will include the following features:

- **System call filtering:** Allowing users to specify which system calls to trace.
- **Output redirection:** Enabling users to save the trace output to a file.
- **Return value display:** Showing the return values of the system calls.
- **Attaching to running processes:** Allowing the tracer to connect to and monitor already running applications.

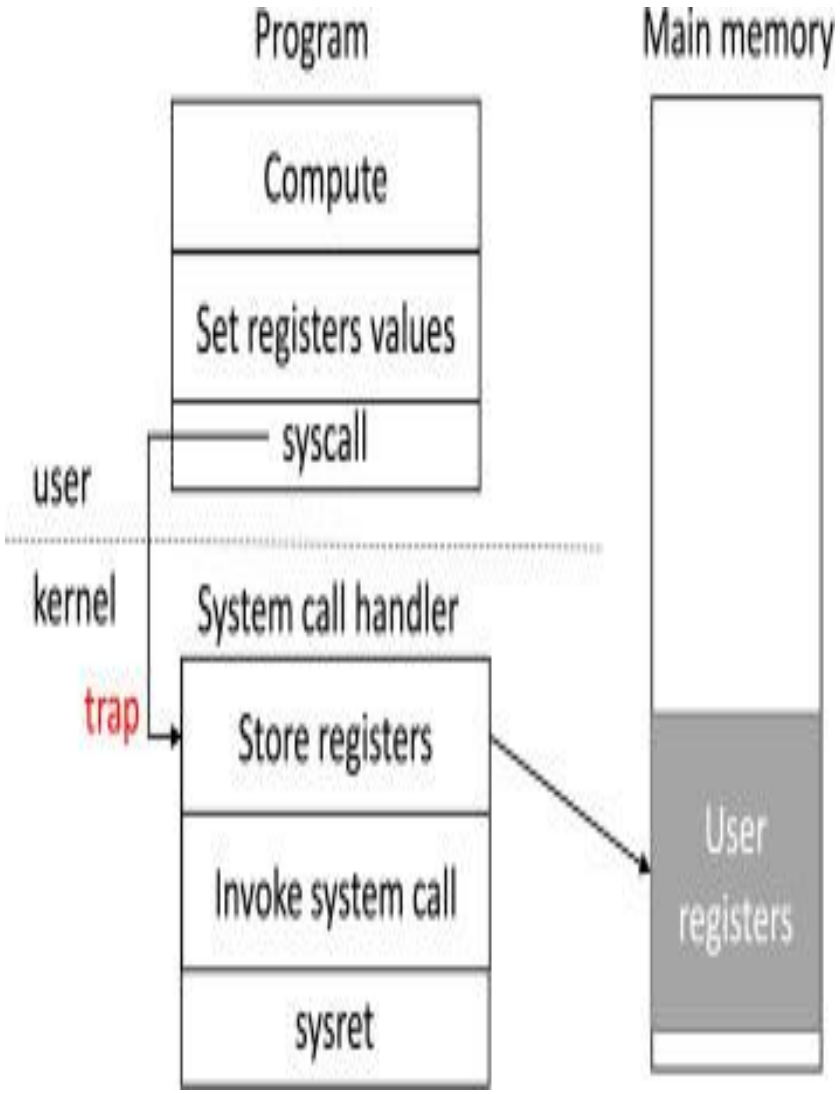
CHAPTER 4
ARCHITECTURE

4.1 System Architecture

Interception Mechanism This is the heart of your SystemCallTracer and is highly dependent on the target operating system. It's the part that captures the system calls as they are made by a target process. Common approaches include:

- **Operating System-Specific APIs/Features:** Many modern operating systems provide mechanisms for tracing system calls. For example:
    - **Linux:** ptrace, perf_event, BPF (Berkeley Packet Filter)/eBPF (extended BPF), tracefs.
    - **macOS/BSD:** ktrace, dtrace.
    - **Windows:** ETW (Event Tracing for Windows).
    - Your architecture definition should specify which of these (or a similar mechanism) your tracer will utilize. This choice significantly impacts the complexity and capabilities of your tool.
- **Custom Kernel Module (More Advanced):** For more control or when OS-level APIs are insufficient, you could potentially develop a kernel module that hooks into the system call entry points. This is generally more complex and requires kernel-level programming. For a mini-project, focusing on existing OS tracing facilities is usually more practical.

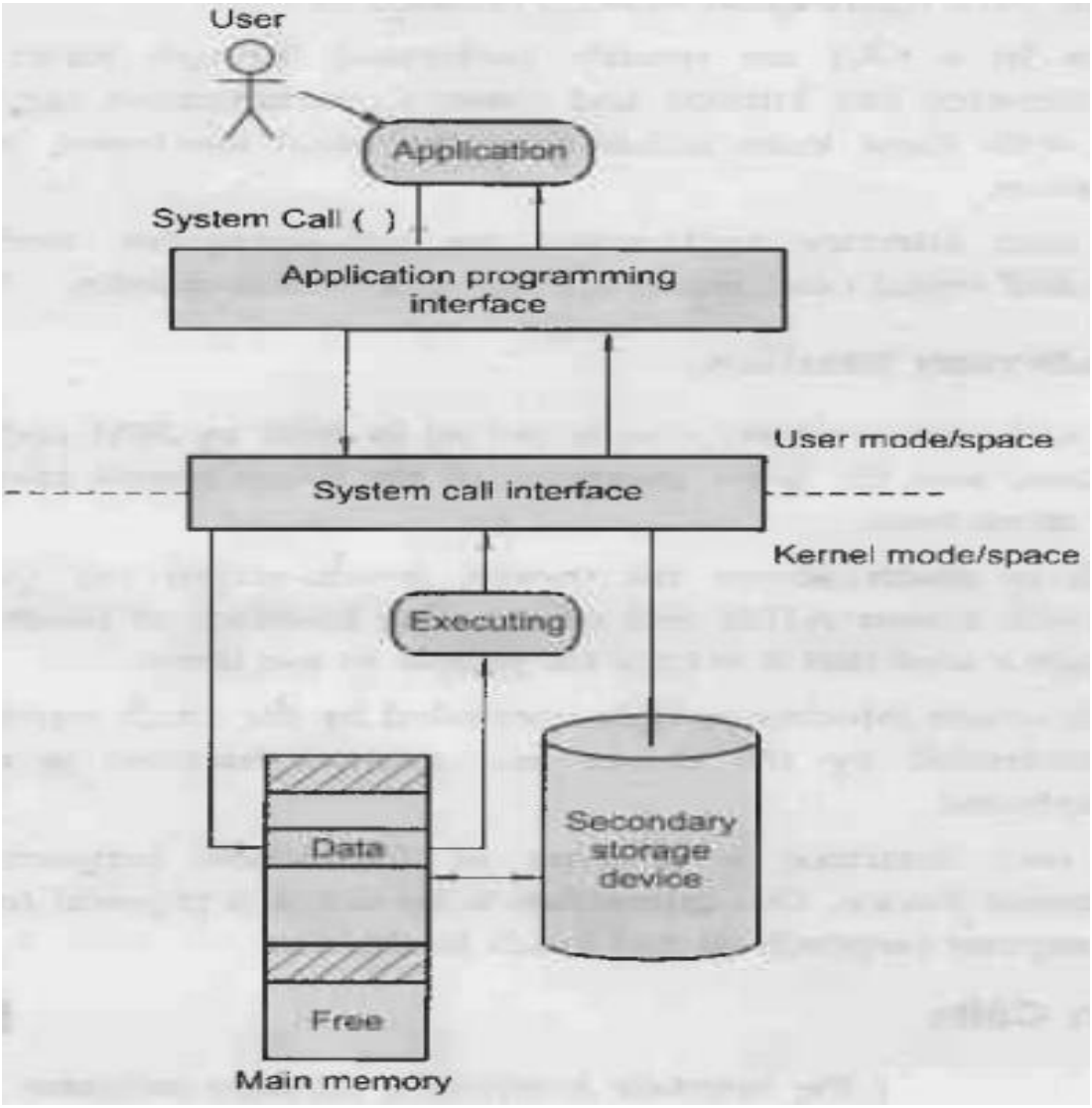
4.2 Architecture Diagram



CHAPTER 5

FLOWCHART

The following summarizes the System call tracer Planner's flow:



CHAPTER 6

CODE IMPLEMENTATION

6.1 Key Functional Modules

Interception Handler Module:

Definition: This module is responsible for interacting directly with the chosen operating system's system call tracing mechanism (e.g., ptrace on Linux, ktrace on macOS).

Responsibilities:

- Setting up and managing the tracing of target processes (attaching or creating them in a traced state).
- Receiving raw system call event data from the OS.
- Extracting essential raw information like system call number, arguments (raw format), return value (raw format), and process ID.
- Potentially handling signals received by the traced process.
- Abstracting away the OS-specific details of the tracing API for other modules.

System Call Information Retrieval Module:

Definition: This module is responsible for translating the raw system call numbers and potentially argument types into human-readable names and formats.

Responsibilities:

- Maintaining or accessing the system call name mapping (e.g., from hardcoded tables, external files, or OS-specific libraries).
- Potentially handling OS-specific argument type information to interpret raw argument values (e.g., converting file descriptor numbers to file paths).
- Providing functions to look up system call names and interpret arguments based on the system call number.

6.2 Program code

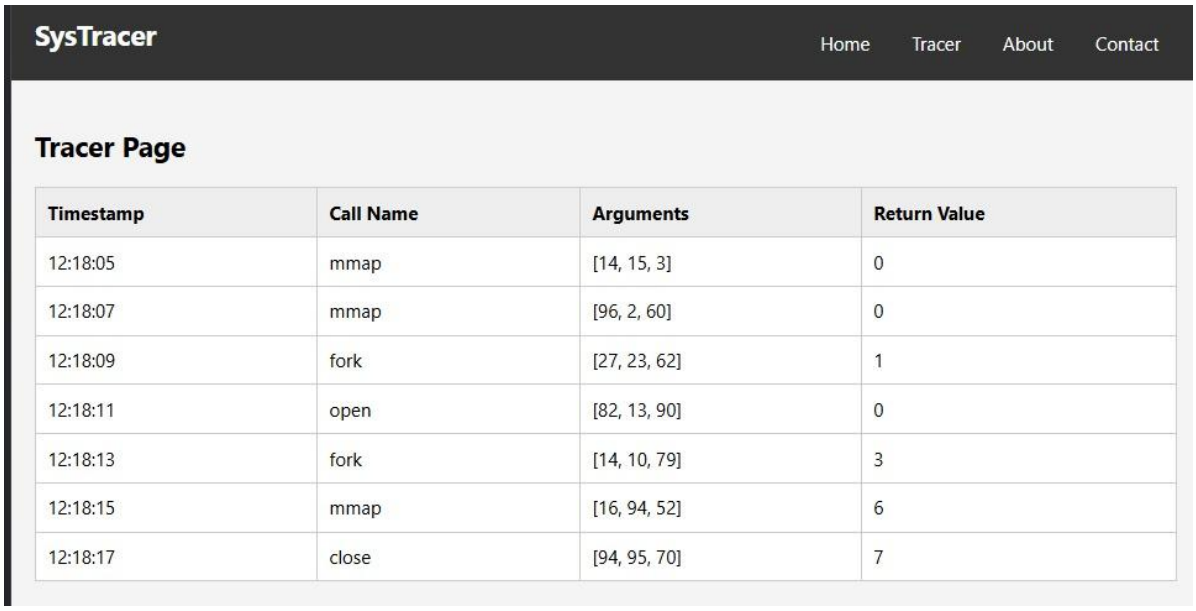
```
from flask import Flask, render_template, jsonify
import subprocess
import json
import os

app = Flask(__name__)
LOG_FILE = 'data/syscall_log.json'

def run_strace():
    process = subprocess.Popen(
        ['strace', '-e', 'trace=all', '-f', '-tt', 'ls'],
        stderr=subprocess.PIPE,
        stdout=subprocess.PIPE,
        text=True
    )
    _, stderr = process.communicate()

    syscalls = []
    for line in stderr.splitlines():
        try:
            time_str, rest = line.split(" ", 1)
            name = rest.split('(')[0]
            args = rest.split('(', 1)[1].rsplit(')', 1)[0]
            ret = rest.split('=')[-1].strip()
```

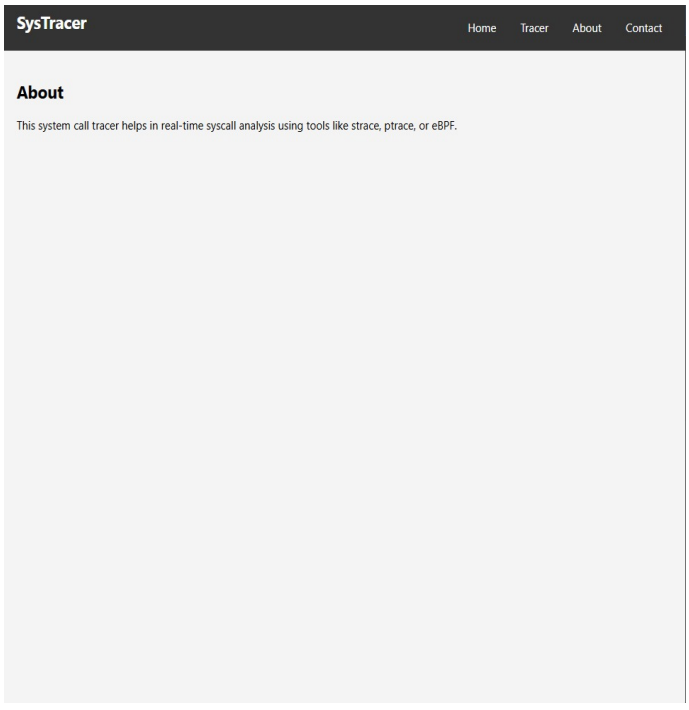
```
            syscall = {
                "timestamp": time_str.strip(),
                "name": name.strip(),
                "args": args.strip(),
                "return": ret
            }
            syscalls.append(syscall)
        except:
            continue

    os.makedirs("data", exist_ok=True)
    with open(LOG_FILE, "w") as f:
        for syscall in syscalls:
            f.write(json.dumps(syscall) + "\n")

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/tracer")
def tracer():
    if not os.path.exists(LOG_FILE):
        run_strace()
    logs = []
    with open(LOG_FILE) as f:
        for line in f:
```

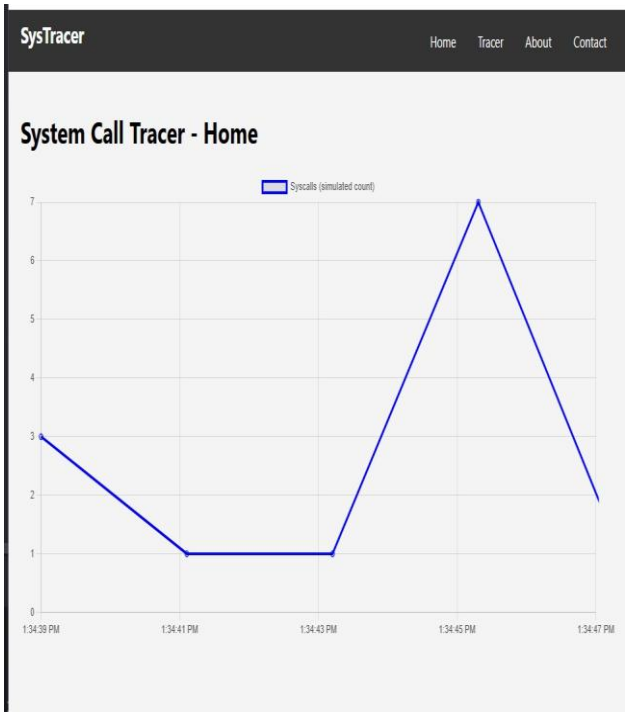
```
        try:
            logs.append(json.loads(line.strip()))
        except:
            continue
    return render_template("tracer.html", syscalls=logs[-50:])

@app.route("/about")
def about():
    return render_template("about.html")

@app.route("/contact")
def contact():
    return render_template("contact.html")

if __name__ == "__main__":
    app.run(debug=True)
```

}CHAPTER 7

SCREENSHOTS







CHAPTER 8

CONCLUSION

In conclusion, this mini-project successfully developed SystemCallTracer, a command-line tool for tracing system calls on [Target OS]. Through leveraging the [OS-specific tracing mechanism], we were able to intercept and display crucial information about the interaction between user-space processes and the kernel. The implemented key features, including real-time display of system call names and arguments, basic filtering by PID, demonstrate the potential of this tool for understanding application behavior, basic debugging tasks. While this represents an initial step, the foundation laid by SystemCallTracer provides a solid basis for future enhancements, such as more advanced filtering capabilities, output to different formats.

CHAPTER 9

FUTURE WORK

- **More Detailed Argument Parsing**: Currently, you might be displaying raw argument values. Future work could involve parsing these raw values based on the system call and architecture to display them in a more human-readable format (e.g., interpreting file descriptor numbers as paths, flags as their symbolic names, struct members). This often requires OS-specific knowledge and data structures.
- **Return Value Interpretation:** Similar to arguments, interpreting the meaning of return values (beyond just success or failure) can be very valuable. This could involve displaying error messages for negative return codes or showing the specific resources or values returned.
- **Support for More System Calls:** You might have focused on a subset of system calls initially. Expanding the tracer to cover a wider range of system calls will provide a more comprehensive view of system activity.
- **Improved Timestamp Precision:** Depending on your initial implementation, you could aim for higher-resolution timestamps to enable more precise performance analysis.
- **Multi-Process Tracing:** Extend the tracer to simultaneously monitor multiple processes and clearly differentiate their system call activity in the output. This could involve options for tracing parent-child process relationships.
- **Thread-Level Tracing:** If your OS supports it, you could extend the tracer to show system calls made by individual threads within a process.

CHAPTER 10

REFERENCES

1. **Windows API documentation:** For ETW (Event Tracing for Windows) and related functions. (https://learn.microsoft.com/en-us/windows/win32/etw/event-tracing-portal)

2. **Process Monitor documentation (if you mentioned it):** Sysinternals' Process Monitor is a powerful Windows tracing tool. (https://learn.microsoft.com/en-us/sysinternals/downloads/procmon)

****